ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

Разработка расширения для Visual Studio Code

Расширение «C++ Runner»

Выполнил(а): _____ Группа: _____

---

1. Цель работы

Изучить, как устроены расширения для Visual Studio Code, и написать собственное расширение на TypeScript, которое помогает при программировании на C++.

2. Описание расширения

Когда решаешь задачи на C++, приходится каждый раз переходить в терминал и вручную набирать команду компиляции и запуска, например `g++ main.cpp -o main && ./main`. Расширение **C++ Runner** делает это одной кнопкой, а также умеет вставлять header guard, создавать заготовку класса и вставлять часто используемые куски кода (сниппеты).

Функция	Как вызвать	Что делает
Скомпилировать и запустить	кнопка ▶ над редактором, кнопка «Run C++» в строке состояния, **Ctrl+Alt+R**	сохраняет файл, компилирует его и запускает программу в терминале
Вставить header guard	правый клик в `.h` / `.hpp` файле	добавляет `#ifndef` / `#define` / `#endif`
Создать класс	правый клик по папке в проводнике	создаёт `Имя.h` и `Имя.cpp` с заготовкой класса
Сниппеты	набрать префикс и нажать Tab	6 шаблонов: `cppmain`, `fori`, `bubblesort` и др.
Настройки	Settings → C++ Runner	выбор компилятора и стандарта C++

3. Создание проекта

Для работы нужны Node.js, VS Code и генератор проектов. Генератор устанавливается командой:

```
npm install -g yo generator-code
```

Проект создаётся командой `yo code`. Ответы на вопросы генератора:

Вопрос	Ответ
What type of extension?	New Extension (TypeScript)
What's the name of your extension?	cpp-runner
What's the identifier?	cpp-runner
What's the description?	Запуск C++ программ одной кнопкой
Initialize a git repository?	Yes

Вопрос	Ответ
Which bundler?	unbundled (без сборщика)
Which package manager?	npm

Сборщик не используется, чтобы проект был проще: TypeScript компилируется обычным компилятором `tsc` в папку `out/`. Генератор создаёт расширение «Hello World» — его команда была заменена тремя своими командами, также добавлены сниппеты и папка с примерами.

4. Структура проекта

```
cpp-runner/
├── .vscode/
│   ├── launch.json        # запуск расширения по F5
│   ├── tasks.json         # компиляция TypeScript в режиме watch
│   ├── settings.json      # настройки редактора для проекта
│   └── extensions.json    # рекомендуемые расширения
├── examples/              # файлы C++ для проверки
│   ├── hello.cpp
│   └── math_utils.h
├── snippets/
│   └── cpp.json           # сниппеты
├── src/
│   ├── extension.ts       # весь код расширения
│   └── test/
│       └── extension.test.ts   # тесты
├── out/                   # скомпилированный JavaScript (создаётся сам)
├── package.json           # манифест расширения
├── tsconfig.json          # настройки TypeScript
├── eslint.config.mjs      # правила проверки кода
├── .gitignore             # что не загружать в git
├── .vscodeignore          # что не класть в .vsix
├── README.md, CHANGELOG.md, LICENSE
```

Весь код находится в одном файле `src/extension.ts`. В `launch.json` добавлен путь к папке `examples`, чтобы при запуске по F5 она сразу открывалась.

5. Файл package.json

`package.json` — это манифест расширения. VS Code читает его ещё до запуска кода и узнаёт из него, какие команды, кнопки, горячие клавиши и настройки нужно добавить.

Поле	Значение	Зачем
`name`, `publisher`	`cpp-runner`, `student`	идентификатор расширения — `student.cpp-runner`
`engines.vscode`	`^1.100.0`	минимальная версия VS Code
`activationEvents`	`onLanguage:cpp`	расширение включается, когда открыт C++ файл
`main`	`./out/extension.js`	файл, с которого начинается работа расширения
`contributes`	объект	что расширение добавляет в редактор

Раздел `contributes` содержит:

- **commands** — три команды. У каждой есть ID (например, `cpp-runner.run`) и название для палитры команд;
- **menus** — где показывать команды: кнопка ▶ в заголовке редактора (`editor/title`) только для `.cpp` файлов, пункт в контекстном меню редактора для `.h` / `.hpp`, пункт в меню папки в проводнике;
- **keybindings** — горячая клавиша `Ctrl+Alt+R` для запуска;
- **configuration** — две настройки: `cppRunner.compiler` и `cppRunner.standard`;
- **snippets** — путь к файлу со сниппетами.

Фрагмент с командами и горячей клавишей:

`package.json → contributes`

```
{
  "commands": [
    {
      "command": "cpp-runner.run",
      "title": "C++ Runner: Скомпилировать и запустить",
      "icon": "$(play)"
    },
    {
      "command": "cpp-runner.insertHeaderGuard",
      "title": "C++ Runner: Вставить header guard"
    },
    {
      "command": "cpp-runner.createClass",
      "title": "C++ Runner: Создать класс"
    }
  ],
  "keybindings": [
    {
      "command": "cpp-runner.run",
      "key": "ctrl+alt+r",
      "when": "editorLangId == cpp"
    }
  ]
}
```

Условие `when` определяет, когда команда доступна. Например, `resourceExtname == .cpp` — открыт файл с расширением `.cpp`.

6. Код расширения

6.1. Активация

При активации VS Code вызывает функцию `activate`. В ней регистрируются команды и создаётся кнопка в строке состояния:

`src/extension.ts`

```
// Вызывается при активации расширения (когда открыт C++ файл или вызвана команда)
export function activate(context: vscode.ExtensionContext) {
    console.log('Расширение "C++ Runner" активировано');

    // Регистрируем команды. ID должны совпадать с contributes.commands в package.json
    const runCommand = vscode.commands.registerCommand('cpp-runner.run', runCurrentFile);
    const guardCommand = vscode.commands.registerCommand('cpp-runner.insertHeaderGuard',
```

```
insertHeaderGuard);
    const classCommand = vscode.commands.registerCommand('cpp-runner.createClass',
createClass);

    statusBarItem = vscode.window.createStatusBarItem(vscode.StatusBarAlignment.Left, 100);
    statusBarItem.text = '$(play) Run C++';
    statusBarItem.tooltip = 'Скомпилировать и запустить текущий файл (Ctrl+Alt+R)';
    statusBarItem.command = 'cpp-runner.run';
    updateStatusBar(vscode.window.activeTextEditor);

    // Когда пользователь переключает вкладку — показываем или прячем кнопку
    const editorListener = vscode.window.onDidChangeActiveTextEditor(updateStatusBar);

    // Всё, что лежит в subscriptions, VS Code сам освободит при деактивации
    context.subscriptions.push(runCommand, guardCommand, classCommand, statusBarItem,
editorListener);
}
```

- `registerCommand` связывает ID команды из `package.json` с функцией, которая её выполняет;
- `createStatusBarItem` создаёт кнопку «▶ Run C++» внизу окна; `$(play)` — встроенная иконка;
- `onDidChangeActiveTextEditor` срабатывает при переключении вкладок — кнопка видна только для `.cpp` файлов;
- всё созданное добавляется в `context.subscriptions`, чтобы VS Code освободил это при выключении расширения.

6.2. Скомпилировать и запустить

src/extension.ts

```
async function runCurrentFile() {
    const editor = vscode.window.activeTextEditor;
    if (!editor || !editor.document.fileName.endsWith('.cpp')) {
        vscode.window.showWarningMessage('Откройте файл .cpp, чтобы его запустить.');
        return;
    }

    // Компилятор читает файл с диска, поэтому сначала сохраняем его
    await editor.document.save();

    // Читаем настройки (Settings -> C++ Runner)
    const config = vscode.workspace.getConfiguration('cppRunner');
    const compiler = config.get<string>('compiler', 'g++');
    const standard = config.get<string>('standard', 'c++17');

    const filePath = editor.document.fileName;     // /Users/.../lab1/main.cpp
    const folder = path.dirname(filePath);         // /Users/.../lab1
    const fileName = path.basename(filePath);      // main.cpp
    const programName = getProgramName(filePath, process.platform); // main или main.exe

    const command = buildRunCommand(compiler, standard, fileName, programName,
vscode.env.shell);

    // Закрываем терминал от прошлого запуска и открываем новый в папке файла
    vscode.window.terminals.find(t => t.name === TERMINAL_NAME)?.dispose();
    const terminal = vscode.window.createTerminal({ name: TERMINAL_NAME, cwd: folder });
    terminal.show();
    terminal.sendText(command);
}
```

Как работает команда:

1. проверяет, что открыт `.cpp` файл, иначе показывает предупреждение;
2. сохраняет файл — компилятор читает его с диска;
3. читает настройки через `getConfiguration('cppRunner')`;
4. закрывает терминал от прошлого запуска и создаёт новый терминал **в папке файла** (параметр `cwd`);
5. отправляет в терминал строку команды через `sendText`.

Строка команды собирается отдельной функцией, потому что у разных терминалов немного разный синтаксис:

src/extension.ts

```
// Собираем строку для терминала: сначала компиляция, и если она успешна — запуск.
// В разных терминалах синтаксис немного отличается.
export function buildRunCommand(compiler: string, standard: string, fileName: string,
    programName: string, shell: string): string {
    const compile = `${compiler} -std=${standard} -Wall "${fileName}" -o "${programName}"`;
    const shellName = shell.toLowerCase();

    if (shellName.includes('powershell') || shellName.includes('pwsh')) {
        // PowerShell: $? — успешно ли выполнилась прошлая команда
        return `${compile}; if ($?) { & ".\\${programName}" }`;
    }
    if (shellName.endsWith('cmd.exe')) {
        return `${compile} && "${programName}"`;
    }
    // bash, zsh (macOS, Linux)
    return `${compile} && "./${programName}"`;
}
```

В macOS и Linux получается, например, `g++ -std=c++17 -Wall "main.cpp" -o "main" && "./main"`. Оператор `&&` запускает программу только если компиляция прошла без ошибок. В PowerShell (Windows) вместо него используется проверка `if ($?)`. Имена файлов взяты в кавычки, чтобы работали пути с пробелами. Программа запускается именно в терминале, потому что ей может понадобиться ввод с клавиатуры (`std::cin`).

Если в коде ошибка, компилятор выведет её в тот же терминал. VS Code распознаёт ссылки вида `main.cpp:5:3`, и по Cmd/Ctrl+клику можно перейти к строке с ошибкой.

6.3. Вставить header guard

Header guard защищает заголовочный файл от повторного подключения: без него, если файл подключён дважды, компилятор выдаст ошибку о повторном объявлении.

src/extension.ts

```
async function insertHeaderGuard() {
    const editor = vscode.window.activeTextEditor;
    if (!editor || !/\.(h|hpp)$/.test(editor.document.fileName)) {
        vscode.window.showWarningMessage('Откройте заголовочный файл .h или .hpp.');
        return;
    }

    const document = editor.document;
    const text = document.getText();
```

```
    const guard = makeGuardName(path.basename(document.fileName));

    if (text.includes('#pragma once') || text.includes(`#ifndef ${guard}`)) {
        vscode.window.showInformationMessage('В файле уже есть header guard.');
        return;
    }

    // Если файл не заканчивается переводом строки — добавим его перед #endif
    const ending = text.length > 0 && !text.endsWith('\n') ? '\n' : '';

    // Обе вставки в одном edit() — их можно отменить одним Ctrl+Z
    await editor.edit(editBuilder => {
        editBuilder.insert(new vscode.Position(0, 0), `#ifndef ${guard}\n#define
${guard}\n\n`);
        editBuilder.insert(document.positionAt(text.length), `${ending}\n#endif //
${guard}\n`);
    });
}
```

src/extension.ts

```
// "math_utils.h" -> "MATH_UTILS_H"
export function makeGuardName(fileName: string): string {
    return fileName.toUpperCase().replace(/[^A-Z0-9]/g, '_');
}
```

Имя макроса строится из имени файла: `math_utils.h` → `MATH_UTILS_H`. Если защита уже есть, файл не меняется. Обе вставки (в начало и в конец) делаются в одном вызове `editor.edit()`, поэтому их можно отменить одним нажатием Ctrl+Z.

6.4. Создать класс

Команда спрашивает имя класса через `showInputBox`. Функция `validateInput` проверяет имя при каждом нажатии клавиши: оно должно начинаться с латинской буквы и не содержать пробелов. Затем через `vscode.workspace.fs.writeFile` создаются два файла, а заголовок открывается в редакторе. Если файлы уже есть, они не перезаписываются.

src/extension.ts (фрагмент createClass)

```
const className = await vscode.window.showInputBox({
    prompt: 'Введите имя класса',
    placeHolder: 'Например: Point',
    validateInput: value => /^[A-Za-z_][A-Za-z0-9_]*$/.test(value)
        ? null
        : 'Имя должно начинаться с латинской буквы и содержать только буквы, цифры и _',
});
```

Пример созданных файлов для класса `Point`:

Point.h

```
#ifndef POINT_H
#define POINT_H

class Point {
public:
    Point();
    ~Point();

private:
};

#endif // POINT_H
```

Point.cpp

```
#include "Point.h"

Point::Point() {
}

Point::~Point() {
}
```

7. Сниппеты

Сниппеты описаны в файле `snippets/cpp.json`. Пример одного сниппета:

snippets/cpp.json

```
{
    "Цикл for по индексу": {
        "prefix": "fori",
        "description": "for (int i = 0; i < n; ++i)",
        "body": [
            "for (int ${1:i} = 0; ${1:i} < ${2:n}; ++${1:i}) {",
            "\t$0",
            "}"
        ]
    }
}
```

`${1:i}` — место для ввода со значением по умолчанию `i` (все поля с номером 1 меняются вместе), `$0` — где окажется курсор в конце. Все сниппеты:

Префикс	Описание
`cppmain`	Заготовка программы: #include <iostream> и функция main
`fori`	for (int i = 0; i < n; ++i)
`readarr`	Функция, которая читает n чисел в обычный массив
`printarr`	Функция, которая печатает массив через пробел
`bubblesort`	Сортировка массива по возрастанию без <algorithm>
`binsearch`	Поиск элемента в отсортированном массиве; возвращает индекс или -1

8. Запуск и проверка

Установка зависимостей и компиляция:

```
npm install
npm run compile
```

Для запуска нужно нажать **F5**. Откроется второе окно VS Code — **Extension Development Host**, в котором работает расширение, с папкой `examples`. Код можно отлаживать: поставить точку останова в `extension.ts` и вызвать команду — выполнение остановится на этой строке.

Проверка работы:

Действие	Результат
открыть `hello.cpp`, нажать ▶, ввести `2 3`	в терминале «C++ Runner» выводится `Sum: 5`
сделать ошибку в коде и нажать ▶	ошибка компилятора выводится в терминал, программа не запускается
открыть `math_utils.h` → «Вставить header guard»	появились `#ifndef MATH_UTILS_H` … `#endif`
правый клик по папке → «Создать класс» → `Point`	созданы `Point.h` и `Point.cpp`
в `.cpp` файле набрать `bubblesort` + Tab	вставлена функция сортировки
в настройках выбрать стандарт `c++20`, нажать ▶	в команде используется `-std=c++20`

Также в проекте есть автоматические тесты (`src/test/extension.test.ts`), они запускаются командой `npm test`. Тесты проверяют: имя header guard; имя программы зависит от ОС; команда запуска для разных терминалов; заготовка класса; команды зарегистрированы.

9. Упаковка и публикация

Расширение упаковывается в установочный файл `.vsix` утилитой `vsce`:

```
npm install -g @vscode/vsce
vsce package
```

Получится файл `cpp-runner-0.0.1.vsix`. Его можно установить через панель Extensions → «...» → Install from VSIX.

Исходный код выложен на GitHub. Папки `node_modules` и `out` в репозиторий не попадают — они указаны в `.gitignore`.

```
git init
git add .
git commit -m "C++ Runner: первая версия"
git branch -M main
git remote add origin https://github.com/USERNAME/cpp-runner.git
git push -u origin main
```

10. Вывод

В ходе работы было создано расширение **C++ Runner** для Visual Studio Code. Я разобрался, как устроен проект расширения, что описывается в `package.json` (команды, меню, горячие клавиши, настройки, сниппеты) и как с помощью VS Code API регистрировать команды, работать с редактором,

терминалом, строкой состояния и файлами. Расширение было проверено в режиме отладки, описано в README, упаковано в `.vsix` и опубликовано на GitHub.